

Breaking Apache's Front Door: CVE-2021-41773 and the Patch That Wasn't Enough

In October 2021, a change meant to fix minor URL-handling edge cases in Apache HTTP Server 2.4.49 instead reopened one of the oldest bugs in web servers: path traversal. Within days, security researchers and then attackers in the wild were using a single crafted HTTP request to read arbitrary files off disk, and in many real deployments, to execute commands. This post walks through what the vulnerability actually is, why it happened, how I reproduced it in an isolated lab, how to detect it, and how it was ultimately fixed — twice.

What the vulnerability is

Apache HTTP Server exposes certain URL paths through directives like `Alias` and `ScriptAlias`, which map a public URL prefix (say, `/cgi-bin/`) to a directory on disk. Before serving a request, Apache is supposed to resolve `.` and `..` segments in the URL and confirm the resulting path is still inside that aliased directory — otherwise a request like `/cgi-bin/../../../../etc/passwd` would obviously escape it.

In 2.4.49, a change to the path-normalization logic broke this guarantee for *URL-encoded* traversal sequences. A request using `%2e` (the URL-encoded form of `.`) instead of a literal dot passed Apache's access-control checks against the alias, because at that point in the request pipeline the string still looked like it was safely inside the aliased directory. The encoded characters were only decoded afterward, when the server actually resolved the filesystem path — by which point it was too late. The request walked straight out of the aliased directory and onto the raw filesystem.

That alone is a path traversal / information disclosure bug (CWE-22): an attacker can read any file the Apache worker process has permission to read, by aiming the traversal at a plain (non-CGI) aliased location so the file is served as-is. But if an aliased directory also has CGI execution enabled (an extremely common

setup for `/cgi-bin/` in 2021), the same trick can be used to traverse onto `/bin/sh` itself. Apache doesn't check "is this actually a CGI script I expect" — it just hands the resolved path to the CGI handler and executes it, feeding it the request body. That upgrades the bug from file disclosure to full, unauthenticated remote code execution as the web server user.

This is tracked as **CVE-2021-41773**, rated High to Critical depending on configuration, and it was confirmed under active exploitation within roughly 72 hours of disclosure.

The patch that wasn't enough

Apache shipped 2.4.50 days later, stripping out the single-encoded `%2e` sequences before the access-control check. That closed the exact proof-of-concept everyone was using — but not the underlying class of bug. Researchers quickly found that **double**-encoding the same characters (`%252e`, which decodes once to `%2e` and a second time to `.`) sailed straight past the new filter, because the filter only accounted for a single decode pass. This bypass is tracked separately as **CVE-2021-42013**, and it's arguably more dangerous than the original: it was published with the CGI/RCE angle already understood, and plenty of admins who'd patched to 2.4.50 believed themselves safe. The real fix didn't land until **2.4.51**.

The lesson generalizes well beyond Apache: patching by pattern-matching a specific encoding of an attack, rather than fixing the ordering of normalization-then-validation, tends to produce exactly this kind of one-step-behind bypass.

Reproducing it

For this assignment I built an isolated Docker lab running the official `httpd:2.4.49` image, with `mod_cgi` enabled, a plain `/static/` alias, and a `/cgi-bin/` alias configured with `Require all granted` — deliberately mirroring how many real production Apache configs looked at the time. Two commands were enough to prove both halves of the impact:

- A crafted `curl` request through the plain `/static/` alias (`/static/.%2e/%2e%2e/%2e%2e/%2e%2e/%2e%2e/etc/passwd`) returned the container's `/etc/passwd` in full, despite `/etc/passwd` having nothing to do with the static directory. A plain alias is used here because the traversed file is served as-is rather than executed.
- A crafted `curl -d "echo; id"` request through the CGI-enabled `/cgi-bin/` alias, pointed at `/bin/sh` instead of a file, returned the output of `id` — proof of code execution, not just disclosure.

I then rebuilt the same configuration on top of `httpd:2.4.51` and reran both requests. Both failed cleanly (a normal 400/404), confirming the fix and isolating it to exactly the base image version — nothing else in the configuration changed between the vulnerable and patched runs.

Detecting it

Detection doesn't have to wait for exploitation. Two independent signals work well together:

1. **Passive version check** — reading the `Server: response` header for `Apache/2.4.49` or `Apache/2.4.50`. This is fast and non-intrusive, but unreliable if the banner has been suppressed (a common hardening step), and it can't tell you whether the vulnerable configuration pattern actually applies.
2. **Active, harmless probe** — sending the same traversal primitive through the plain alias but pointed at something benign, like Apache's own `httpd.conf`, rather than sensitive data. If the response contains recognizable config content (e.g., the `ServerRoot` directive), the host is confirmed exploitable regardless of what the banner claims.

Combining both gives a low-false-positive signal suitable for a vulnerability scanner or a quick pre-deployment check, without needing to actually pull `/etc/passwd` or spawn a shell in a production system.

Remediation and takeaways

The fix here is simple to state and non-negotiable: **upgrade to Apache 2.4.51 or later**. 2.4.50 is not sufficient. Beyond the immediate patch, this CVE is a good case study in a few broader principles:

- **Order of operations matters in security checks.** Normalizing untrusted input *before* validating it, and never re-normalizing it again afterward, avoids this entire bug class.
- **Defense in depth still pays off.** Deployments with a blanket `Require all denied` on `<Directory />` were meaningfully less exposed even before patching, because the traversed path still hit that deny rule.
- **A patch closing the disclosed PoC isn't the same as closing the vulnerability class.** The 2.4.50 to 2.4.51 gap is a textbook example of why re-testing a fix against variations of the original attack, not just the original request, matters before calling something "resolved."

Sources

- Apache HTTP Server Security Advisory: https://httpd.apache.org/security/vulnerabilities_24.html
- NVD — CVE-2021-41773: <https://nvd.nist.gov/vuln/detail/CVE-2021-41773>
- NVD — CVE-2021-42013: <https://nvd.nist.gov/vuln/detail/CVE-2021-42013>
- Vulnerability reported by Ash Daulton and the cPanel Security Team (per Apache's advisory).